Numerical Linear Algebra (2MBC30).

Project 2 - Group 7

AI aknowledgment.

AI was used for assistance with LaTeX formatting in overleaf. Concretely, we used it to present our MATLAB code as nice code boxes that distinguish themselves from ordinary text and to help us place our graphs/figures/tables into the document in the right position with the right format.

1 Introduction

An important application of linear algebra is processing images to identify objects or underlying structures within them. However, this presents problems, since different images may represent the same object. Therefore, methods are needed that recognize underlying structures rather than simply memorize individual examples.

This motivates our report, in which we investigate how linear algebra can be used to identify handwritten digits (from 0 to 9). We use the US Postal Service database, which contains various handwritten examples of each digit, to study this classification problem.

We consider three methods for predicting the digit in an unseen image: the nearest-average method, which represents each digit class by a single average vector; the nearest-neighbor method, which compares the new image with every training image; and a method based on singular value decomposition (SVD). Finally, we compare their accuracy and analyze the errors they produce.

2 Data and setup.

The data from the US portal service database is composed of various 16×16 grayscale images of hanwritten digits from 0 to 9 (hence even though two images represent the same digit, they are not identical mathematically). Each pixel holds a real number between -1 and 1 representing brightness level. Specifically, if the pixel is -1 is pure white, 1 is pure black and intermediate values represent grays, with increasing darkness as the value increases.

We store each 16×16 grayscale image as a vector in $\mathbb{R}^{256}$ and then store all these vectors in a matrix. In total we have 1707 training images and 2007 testing images. We stack all the training images in a matrix $X \in \mathbb{R}^{256\times 1707}$ such that column i of the matrix given by $x_i \in \mathbb{R}^{256}$ represents one training image, and denote the true number label of a traning image x_i by y_i (this means that if the vector x_i represents for instance the digit 2, then $y_i = 2$)

Similarly we stack all test images onto a matrix $Z \in$ such that column ℓ of the matrix given by $z_\ell \in \mathbb{R}^{256}$ represents one test image, and denote the true number label of a test image z_ℓ by u_ℓ. We use these test labels to give predictions a score.

We also define a function $d : \mathbb{R}^{256} \times \mathbb{R}^{256} \to \mathbb{R}$ which represents how closely aligned two images are, using the Euclidean norm, so

$$d(x, z) = ||x - z||_2 = \sqrt{\sum_{k=1}^{256} (x_k - z_k)^2}$$

Finally a key detail to point out is that our data holds an important limitation, as the distribution of the digits is more even. The table below shows this distibution:

Digit	0	1	2	3	4	5	6	7	8	9	Total
Training	319	252	202	131	122	88	151	166	144	132	1707
Training (%)	18.7	14.8	11.8	7.7	7.1	5.2	8.8	9.7	8.4	7.7	100
Test	359	264	198	166	200	160	170	147	166	177	2007
Test (%)	17.9	13.2	9.9	8.3	10.0	8.0	8.5	7.3	8.3	8.8	100

Table 1: Distribution of handwritten digits in the training and test datasets.

Notice for example that the digit 0 holds a higher % of the total distribution of traning images, thus this means that

our algorithm has many more examples of how people write a 0 than how they write a 5, for example. Consequently, it might recognize zeros more accurately than fives, which is a limitation.

3 Methods

For each digit $j \in \{0, ..., 9\}$, let $I_j = \{i : y_i = j\}$ be the indices of its training images and $n_j = |I_j|$.Each method assigns a predicted label $\hat{y}(z)$ to a test image $z \in \mathbb{R}^{256}$.

3.1 Nearest average

Each digit is represented by its mean training image, and a test image gets the label of the closest mean:

$$\mu_j = \frac{1}{n_j} \sum_{i \in I_j} x_i \quad \hat{y}_{avg}(z) = \arg \min_{j \in \{0,...,9\}} d(z, \mu_j)$$

The mean is the natural single representative of a class, since it minimizes $\sum_{i \in I_j} ||x_i - m||_2^2$ over all $m \in \mathbb{R}^{256}$. However, averaging merges different writing styles. Thus, a class containing both slanted and upright versions of a digit is represented by one blurred intermediate shape. This method is very cheap. The model consists of only 10 vectors, and classifying an image requires 10 distance computations.

3.2 Nearest neighbor

All training images are kept, and a test image gets the label of the closest one:

$$i^*(z) = \arg \min_{i=1,...,1707} d(z, x_i), \quad \hat{y}_{NN}(z) = y_{i^*(z)}$$

Unlike the previous method, this preserves the individual handwriting styles in the training set. The price is that every prediction compares z with all 1707 training images, and the full training set(256×1707 values) must be stored. Moreover, a single atypical or badly written training image can determine the result. A natural extension of this method is the k-nearest-neighbor method, which takes a majority vote among the k closest training images.

3.3 SVD subspace classifier

For each digit j, collect its training images and compute the economy SVD:

$$A_j = (x_i)_{i \in I_j} \in \mathbb{R}^{256 \times n}, \qquad A_j = U_j \Sigma_j V_j^T$$

Since the singular values are ordered $\sigma_{j,1} \geq \sigma_{j,2} \geq ... \geq 0$, the first columns of U_j capture the dominant variations in how digit j is written. We keep the first $r = 15$ of them in $Q_j = (u_{j,1}|...|u_{j,r})$.Because $Q_j^T Q_j = I$, the normal equations give the best approximation of z in $R(Q_j)$ directly:

$$c^* = \arg \min_{c \in \mathbb{R}^r} ||z - Q_j c||_2 = Q_j^T z$$

and z is assigned to the digit whose subspaces reconstructs it the best:

$$\hat{y}_{SVD}(z) = \arg \min_{j=0,...,9} ||z - Q_j(Q_j^T z)||_2$$

We chose r by testing several values. A small r cannot capture all handwriting styles, while a large r makes the subspaces of different digits overlap. After the ten SVDs are computed once, each prediction needs only ten products $Q_j^T z$.

4 Code implementation and results

4.1 Code

In this section we provide the matlab code we used for the computation. Our code first loads the training and test data and then applies the different algorithms discussed in chapter 3. After applying each method on the test images, the code prints the correct counts, accuracies, runtimes, and first five misclassified indices. We also count correct predictions for each true digit and print the correct count, total count, and accuracy for digits 0 through 9 after each method.

The code used was the following

```
clear all
close all

% Read the data for this problem
load digitimagesandlabels

% Count the test images for each digit
test_images_per_digit = zeros(10, 1);
for digit = 0 : 9
    test_images_per_digit(digit + 1) = sum(testimageslabels == digit);
end


% Nearest Average

disp('Starting Method 1: Nearest Average...');
tic;

% Train
matrixofaveragenumbers = zeros(256, 10);
for j = 0 : 9
    numberofnumberj = 0;
    sumofnumberj = zeros(256,1);
    for i = 1:1707
      if trainingimageslabels(i) == j
        % Image i is labeled as the digit j
        numberofnumberj = numberofnumberj + 1;
        sumofnumberj = sumofnumberj + trainingimages(:,i);
      end
    end
    % Plot the average
    averagenumberj = sumofnumberj / numberofnumberj;

    matrixofaveragenumbers(:,j+1) = averagenumberj;
end

% Test
correct_predictions_an = 0;
correct_predictions_per_digit_an = zeros(10, 1);
failed_an_indices = [];
for j = 1 : 2007
    d_min = Inf;
    closestnumber = -1;
    for i = 0 : 9
        d = norm(testimages(:, j) - matrixofaveragenumbers(:, i+1));
        if d < d_min
            d_min = d;
            closestnumber = i;
        end
    end
    if closestnumber == testimageslabels(j)
        correct_predictions_an = correct_predictions_an + 1;
        digit_index = double(testimageslabels(j)) + 1;
        correct_predictions_per_digit_an(digit_index) = correct_predictions_per_digit_an(
    digit_index) + 1;
    else
```

```
        failed_an_indices = [failed_an_indices, j];
    end
end
time_an = toc;

fprintf('First 5 misclassified images for Nearest Average: %s\n', num2str(failed_an_indices(1:5)));
accuracy_an = (correct_predictions_an / 2007) * 100;
fprintf('Total Correct: %d out of 2007\n', correct_predictions_an);
fprintf('Accuracy: %.2f%%\n', accuracy_an);
fprintf('Execution time: %.2f seconds\n\n', time_an);

% Print the accuracy for each digit
accuracy_per_digit_an = zeros(10, 1);
fprintf('Accuracy per digit:\n');
for digit = 0 : 9
    digit_index = digit + 1;
    if test_images_per_digit(digit_index) > 0
        accuracy_per_digit_an(digit_index) = (correct_predictions_per_digit_an(digit_index) / test_images_per_digit(digit_index)) * 100;
        fprintf('Digit %d: %d out of %d correct, Accuracy: %.2f%%\n', digit, correct_predictions_per_digit_an(digit_index), test_images_per_digit(digit_index), accuracy_per_digit_an(digit_index));
    else
        fprintf('Digit %d: No test images\n', digit);
    end
end
fprintf('\n');



% Nearest Neighbor

disp('Starting Method 2: Nearest Neighbor...');
tic;

% method's accuracy.

correct_predictions_nn = 0;
correct_predictions_per_digit_nn = zeros(10, 1);
failed_nn_indices = [];
for j = 1 : 2007
    d_min = Inf;
    closestnumber = -1;
    for i = 1 : 1707
        d = norm(testimages(:, j) - trainingimages(:, i));
        if d < d_min
            d_min = d;
            closestnumber = trainingimageslabels(i);
        end
    end
    if closestnumber == testimageslabels(j)
        correct_predictions_nn = correct_predictions_nn + 1;
        digit_index = double(testimageslabels(j)) + 1;
        correct_predictions_per_digit_nn(digit_index) = correct_predictions_per_digit_nn(digit_index) + 1;
    else
        failed_nn_indices = [failed_nn_indices, j];
    end
end
time_nn = toc;

fprintf('First 5 misclassified images for Nearest Neighbor: %s\n', num2str(failed_nn_indices(1:5)));
accuracy_nn = (correct_predictions_nn / 2007) * 100;
fprintf('Total Correct: %d out of 2007\n', correct_predictions_nn);
fprintf('Accuracy: %.2f%%\n', accuracy_nn);
fprintf('Execution time: %.2f seconds\n\n', time_nn);

% Print the accuracy for each digit
accuracy_per_digit_nn = zeros(10, 1);
```

```
fprintf('Accuracy per digit:\n');
for digit = 0 : 9
    digit_index = digit + 1;
    if test_images_per_digit(digit_index) > 0
        accuracy_per_digit_nn(digit_index) = (correct_predictions_per_digit_nn(digit_index) /
    test_images_per_digit(digit_index)) * 100;
        fprintf('Digit %d: %d out of %d correct, Accuracy: %.2f%%\n', digit,
    correct_predictions_per_digit_nn(digit_index), test_images_per_digit(digit_index),
    accuracy_per_digit_nn(digit_index));
    else
        fprintf('Digit %d: No test images\n', digit);
    end
end
fprintf('\n');



% SVD

disp('Starting Method 3: SVD...');
tic;

r = 15;
% by testing we found that r = 15 has the best accuracy
Ur = cell(10, 1);
for i = 0:9
    A_j = trainingimages(:, trainingimageslabels == i);
    [U, S, V] = svd(A_j, 'econ');
    Ur{i + 1} = U(:, 1:r);
end

correct_predictions_svd = 0;
correct_predictions_per_digit_svd = zeros(10, 1);
failed_svd_indices = [];
for i = 1:2007
    test_image = testimages(:, i);
    d_min = Inf;
    closest_number = -1;
    for j = 0:9
        x_proj = Ur{j + 1} * (Ur{j + 1}' * test_image);
        d = norm(test_image - x_proj);
        if d < d_min
            d_min = d;
            closest_number = j;
        end
    end
    if closest_number == testimageslabels(i)
        correct_predictions_svd = correct_predictions_svd + 1;
        digit_index = double(testimageslabels(i)) + 1;
        correct_predictions_per_digit_svd(digit_index) = correct_predictions_per_digit_svd(
    digit_index) + 1;
    else
        failed_svd_indices = [failed_svd_indices, i];
    end
end
time_svd = toc;

fprintf('First 5 misclassified images for SVD: %s\n', num2str(failed_svd_indices(1:5)));
accuracy_svd = (correct_predictions_svd / 2007) * 100;
fprintf('Total Correct: %d out of 2007\n', correct_predictions_svd);
fprintf('Accuracy: %.2f%%\n', accuracy_svd);
fprintf('Execution time: %.2f seconds\n\n', time_svd);

% Print the accuracy for each digit
accuracy_per_digit_svd = zeros(10, 1);
fprintf('Accuracy per digit:\n');
for digit = 0 : 9
    digit_index = digit + 1;
    if test_images_per_digit(digit_index) > 0
        accuracy_per_digit_svd(digit_index) = (correct_predictions_per_digit_svd(digit_index) /
    test_images_per_digit(digit_index)) * 100;
```

```
        fprintf('Digit %d: %d out of %d correct, Accuracy: %.2f%%\n', digit,
    correct_predictions_per_digit_svd(digit_index), test_images_per_digit(digit_index),
    accuracy_per_digit_svd(digit_index));
    else
        fprintf('Digit %d: No test images\n', digit);
    end
end
fprintf('\n');
```

4.2 Results

The results we obtained are summarized in the tables below:

Table 2: Overall classification results on the 2,007 test images.

Method	Correct	Incorrect	Accuracy	Time (s)
Nearest average	1623	384	80.87%	0.02
Nearest neighbor	1838	169	91.58%	1.00
SVD ($r = 15$)	1889	118	94.12%	0.08

Table 3: Results by true digit. Each method's entry gives the number correctly classified and the accuracy for that digit.

Digit	Test images	Nearest average	Nearest neighbor	SVD ($r = 15$)
0	359	297 (82.73%)	354 (98.61%)	355 (98.89%)
1	264	259 (98.11%)	258 (97.73%)	260 (98.48%)
2	198	144 (72.73%)	175 (88.38%)	179 (90.40%)
3	166	131 (78.92%)	148 (89.16%)	148 (89.16%)
4	200	146 (73.00%)	169 (84.50%)	186 (93.00%)
5	160	114 (71.25%)	131 (81.88%)	140 (87.50%)
6	170	144 (84.71%)	157 (92.35%)	162 (95.29%)
7	147	120 (81.63%)	138 (93.88%)	141 (95.92%)
8	166	128 (77.11%)	141 (84.94%)	153 (92.17%)
9	177	140 (79.10%)	167 (94.35%)	165 (93.22%)
Total	2007	1623 (80.87%)	1838 (91.58%)	1889 (94.12%)

Table 4: First five misclassified test-image indices for each method. Indices start at one, following MATLAB conventions.

Method	First five misclassified indices
Nearest average	2, 4, 13, 16, 23
Nearest neighbor	4, 13, 14, 18, 28
SVD ($r = 15$)	4, 13, 28, 54, 88

We discuss them in the following section

5 Discussion and conclusion.

The results in Tables 2 and 3 show important differences in the performance of the three methods. First, we can see that the nearest average method was the least accurate overall and for every digit except 1, a digit for in which all methods score exceptionally high. We can see that in general the SVD method was the most accurate both overall and almost for every digit, followed by the nearest neighbor method.

Since the SVD and nearest neighbor methods account for variations in handwriting more strongly, the results

suggest that representing each digit by a single average image is less effective than methods that account for these variations. Concretely, since different people write the same digit in different ways, averaging these images may remove important features which come from this variation that distinguish one digit from another. On the other hand, the nearest-neighbor method compares each test image directly with all training images, allowing it to recognize different handwriting styles.

The confusion matrices in Figure 1 help us visualize the improvements that the nearest neighbor and SVD methods bring. In a confusion matrix correct predictions lie on the diagonal, and a darker color means that a higher number of the test samples were predicted as that digit. Here we can see that for all three methods the diagonal entries are all much darker than the other entries, however, we can clearly see that the nearest average method has a lighter diagonal than the nearest neighbor method and SVD, and contains a higher number of errors, which supports the previous point we made

Furthermore, Table 5 shows us that the errors of the nearest average method are mainly from two pairs: 38 zeros are classified as 6s and 32 fours as 9s. These digits have similar shapes (a 0 and a 6 both consist mainly of a closed circle, and a 4 and a 9 differ mostly in whether the top is closed), so after averaging their mean images potentially make the 2 digits hard to distinguish. Both nearest neighbor and SVD reduce these errors significantly (SVD makes only 5 errors for $4 \to 9$), which shows that these methods can distinguish between similar digits more effectively.

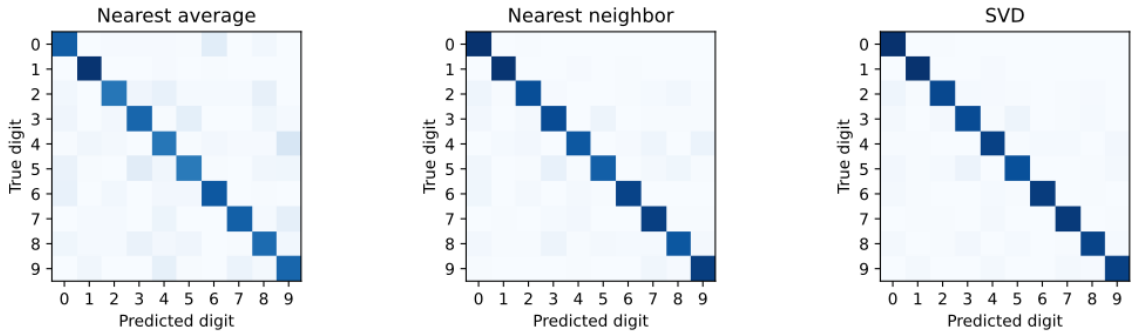


Figure 1: Confusion matrices of the three methods. A darker colour means more samples were predicted under that combination.

Table 5: Most frequent errors on the 2007 test images.

Method	Most frequent errors
Nearest average	$0 \to 6$ (38), $4 \to 9$ (32), $5 \to 3$ (18), $2 \to 8$ (16), $9 \to 4$ (15)
Nearest neighbor	$5 \to 3$ (12), $4 \to 9$ (12), $4 \to 7$ (9), $2 \to 0$ (9), $8 \to 3$ (8)
SVD ($r = 15$)	$5 \to 3$ (9), $2 \to 0$ (9), $3 \to 5$ (8), $4 \to 9$ (5), $9 \to 7$ (4)

The SVD method performed best overall, as it correctly classified an impressive 94% of all images which was the highest of the three methods. It is also the most accurate option for nearly all digits. A possible explanation is that by representing each digit using a 15-dimensional subspace, the SVD captures important patterns and variations within each digit class better than by averaging or using one individual image. Furthermore, the SVD method's runtime was significantly shorter than the nearest metho runtime and only slightly longer than the nearest average method, suggesting that it's the best fitted option to balance accuracy and efficiency.

Looking at the results for individual digits, we observe that some digits are consistently easier to classify than others. For example, the SVD method recognized digits 0 and 1 with accuracies of 98.89% and 98.48%, respectively, whereas digits 5 and 3 achieved only 87.50% and 89.16%. A possible explanation for this discrepancy is that certain digits have more consistent shapes, while others share similar features with different digits, making them harder to distinguish. Furthermore, the unequal distribution of training images may also influence these differences. For

instance, the training set contains 319 examples of digit 0 but only 88 examples of digit 5, potentially giving the algorithms a better representation of the different ways in which zeros are written.

Finally, considering Table 4, we can see that the misclassified indices reveals that test images 4 and 13 were incorrectly classified by all three methods. This suggests that some images may be particularly difficult to recognize, regardless of the classification method used.

Overall, the results demonstrate the effectiveness of linear algebra in handwritten-digit recognition. In particular, SVD provides the best balance between accuracy and execution time among the three methods tested, illustrating how representing images using lower-dimensional subspaces can be useful for classification.